

Lab 2: C Programming Basics

Lab Report and Knowledge Check

I. INTRODUCTION

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure, compilation stages, and basic Linux commands for software development.

II. C PROGRAM STRUCTURE AND EXAMPLES

A. Basic Hello World Program

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello, World!\n");
5     return 0;
6 }
```

Listing 1. Basic Hello World Program

B. Program with User Input

This example demonstrates the use of printf and scanf for input/output operations:

```
1 #include <stdio.h>
2
3 int main() {
4     int number;
5
6     printf("Enter a number: ");
7     scanf("%d", &number);
8
9     printf("You entered: %d\n", number);
10    return 0;
11 }
```

Listing 2. Program with User Input

C. Program with Formatted Output

Demonstrating multiple format specifiers and variables:

```
1 #include <stdio.h>
2
3 int main() {
4     char name[50];
5     int age;
6     float height;
7
8     printf("Enter your name: ");
9     scanf("%s", name);
10
11    printf("Enter your age: ");
12    scanf("%d", &age);
13 }
```

```
14 printf("Enter your height (m): ");
15 scanf("%f", &height);
16
17 printf("\n--- User Info ---\n");
18 printf("Name: %s\n", name);
19 printf("Age: %d years\n", age);
20 printf("Height: %.2f m\n", height);
21
22 return 0;
23 }
```

Listing 3. Program with Formatted Output

III. GCC COMPILATION PROCESS

A. Installing GCC on Linux

The following commands are used to install and verify GCC:

```
1 # Install build-essential package
2 sudo apt install build-essential
3
4 # Check GCC version
5 gcc --version
```

Listing 4. GCC Installation Commands

B. Four Stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

TABLE I
THE FOUR STAGES OF C COMPILATION

Stage	Command	Output	Purpose
1. Preprocessing	gcc -E hello.c -o hello.i	.i file	Expand headers & macros
2. Compiling	gcc -S hello.i -o hello.s	.s file	Generate Assembly code
3. Assembling	gcc -c hello.s -o hello.o	.o file	Create object file
4. Linking	gcc hello.o -o hello	executable	Final executable

C. COMPILATION STAGE EXAMPLES

1) Stage 1: Preprocessing:

```
1 # Expand all headers and macros
2 gcc -E hello.c -o hello.i
3
4 # View the preprocessed output
5 cat hello.i | less
```

Listing 5. Preprocessing Stage

2) Stage 2: Compilation to Assembly:

```
1 # Generate assembly code
2 gcc -S hello.i -o hello.s
3
4 # View assembly code
5 cat hello.s
```

Listing 6. Compilation Stage

3) Stage 3: Assembly to Object File:

```
1 # Create object file (machine code)
2 gcc -c hello.s -o hello.o
3
4 # Check file type
5 file hello.o
```

Listing 7. Assembly Stage

4) Stage 4: Linking:

```
1 # Create final executable
2 gcc hello.o -o hello
3
4 # Run the program
5 ./hello
```

Listing 8. Linking Stage

IV. RETURN VALUES AND EXIT STATUS

A. Understanding Return Values

The return value of `main()` indicates program success or failure to the OS:

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Program executed successfully\n");
5     return 0; // 0 indicates success
6 }
```

Listing 9. Program with Return Value

B. Checking Exit Status

```
1 # Run program
2 ./hello
3
4 # Check exit status (returns 0 for success)
5 echo $?
6
7 # Try with return 1
8 # Modify program to return 1, recompile, and check:
9 echo $? # Should return 1 (indicating error)
```

Listing 10. Checking Program Exit Status

V. KNOWLEDGE CHECK QUESTIONS

The following questions assess understanding of Lecture 2 concepts:

A. Question 1

Q: What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

B. Question 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

C. Question 3

Q: What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

D. Question 4

Q: Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the user

E. Question 5

Q: What GCC command would you use to:

- 1) Only preprocess a file without compiling?
- 2) Generate assembly code but not assemble it?
- 3) Create an object file without linking?

F. Question 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

VI. LAB TASKS COMPLETED

- 1) **Task 1:** Compiled `hello.c` with return 0 and verified exit status using `echo $?`
- 2) **Task 2:** Modified program to return 1 and observed different exit status
- 3) **Task 3:** Created program accepting user input and displaying it
- 4) **Task 4:** Executed all four compilation stages separately and documented output files
- 5) **Task 5:** Compiled and ran program returning integer value 50

VII. SCREENSHOTS AND EVIDENCE

Note: Students should include screenshots of:

- Terminal showing GCC installation and version check
- Each compilation stage (.i, .s, .o files)
- Program execution with different return values
- Exit status verification using `echo $?`

VIII. CONCLUSION

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.

GITHUB REPOSITORY

All source code, compilation outputs, and this report have been submitted to the course GitHub repository:

Note to Students: Remember to push all your code files and well-documented report to your personal GitHub repository and the c4mpus task portal before the deadline.